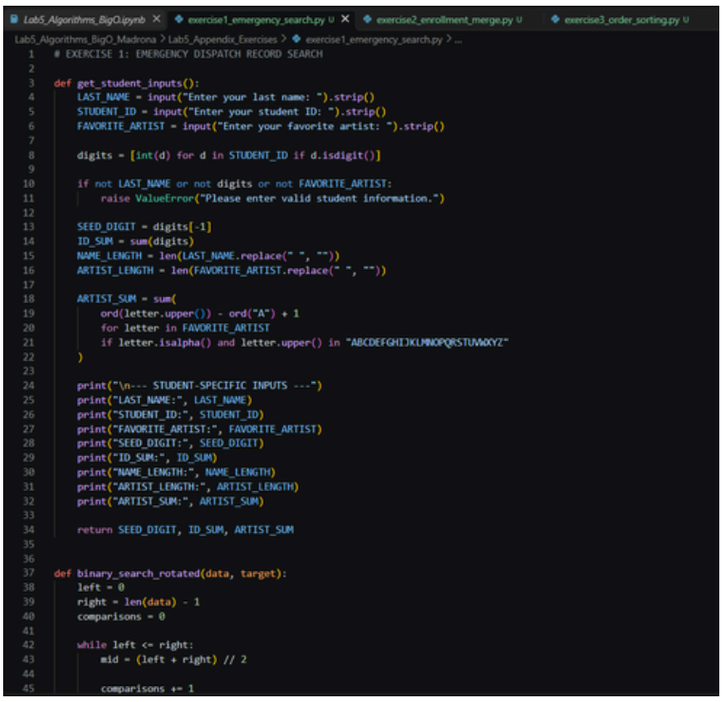
MADRONA, Glydel B.
BSECE-1C

ACTIVITY 5 - ALGORITHMS AND BIG O

EXERCISE 1: EMERGENCY DISPATCH RECORD SEARCH



```python
# EXERCISE 1: EMERGENCY DISPATCH RECORD SEARCH

def get_student_inputs():
    LAST_NAME = input("Enter your last name: ").strip()
    STUDENT_ID = input("Enter your student ID: ").strip()
    FAVORITE_ARTIST = input("Enter your favorite artist: ").strip()

    digits = [int(d) for d in STUDENT_ID if d.isdigit()]

    if not LAST_NAME or not digits or not FAVORITE_ARTIST:
        raise ValueError("Please enter valid student information.")

    SEED_DIGIT = digits[-1]
    ID_SUM = sum(digits)
    NAME_LENGTH = len(LAST_NAME.replace(" ", ""))
    ARTIST_LENGTH = len(FAVORITE_ARTIST.replace(" ", ""))

    ARTIST_SUM = sum(
        ord(letter.upper()) - ord("A") + 1
        for letter in FAVORITE_ARTIST
        if letter.isalpha() and letter.upper() in "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    )

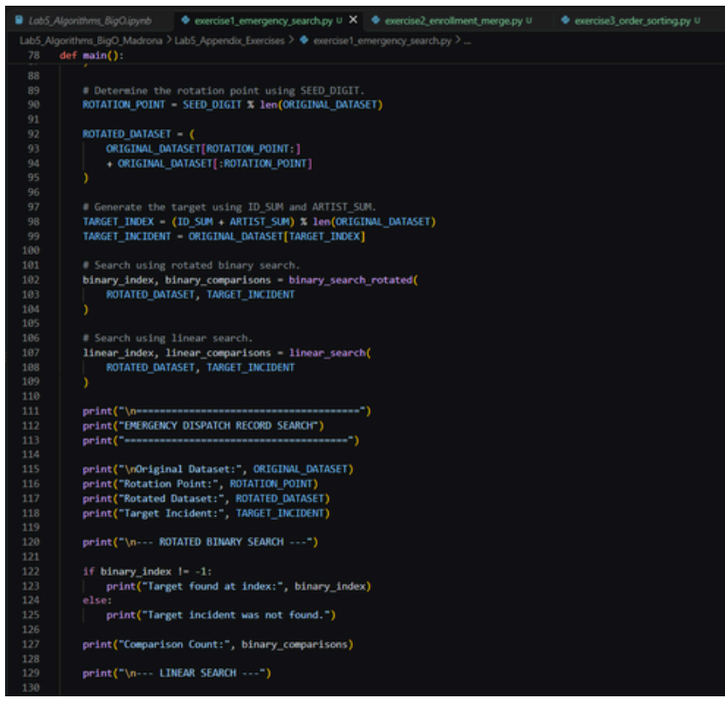
    print("\n--- STUDENT-SPECIFIC INPUTS ---")
    print("LAST_NAME:", LAST_NAME)
    print("STUDENT_ID:", STUDENT_ID)
    print("FAVORITE_ARTIST:", FAVORITE_ARTIST)
    print("SEED_DIGIT:", SEED_DIGIT)
    print("ID_SUM:", ID_SUM)
    print("NAME_LENGTH:", NAME_LENGTH)
    print("ARTIST_LENGTH:", ARTIST_LENGTH)
    print("ARTIST_SUM:", ARTIST_SUM)

    return SEED_DIGIT, ID_SUM, ARTIST_SUM


def binary_search_rotated(data, target):
    left = 0
    right = len(data) - 1
    comparisons = 0

    while left <= right:
        mid = (left + right) // 2

        comparisons += 1
```



```python
def main():
    '

    # Determine the rotation point using SEED_DIGIT.
    ROTATION_POINT = SEED_DIGIT % len(ORIGINAL_DATASET)

    ROTATED_DATASET = (
        ORIGINAL_DATASET[ROTATION_POINT:]
        + ORIGINAL_DATASET[:ROTATION_POINT]
    )

    # Generate the target using ID_SUM and ARTIST_SUM.
    TARGET_INDEX = (ID_SUM + ARTIST_SUM) % len(ORIGINAL_DATASET)
    TARGET_INCIDENT = ORIGINAL_DATASET[TARGET_INDEX]

    # Search using rotated binary search.
    binary_index, binary_comparisons = binary_search_rotated(
        ROTATED_DATASET, TARGET_INCIDENT
    )

    # Search using linear search.
    linear_index, linear_comparisons = linear_search(
        ROTATED_DATASET, TARGET_INCIDENT
    )

    print("\n========================================")
    print("EMERGENCY DISPATCH RECORD SEARCH")
    print("========================================")

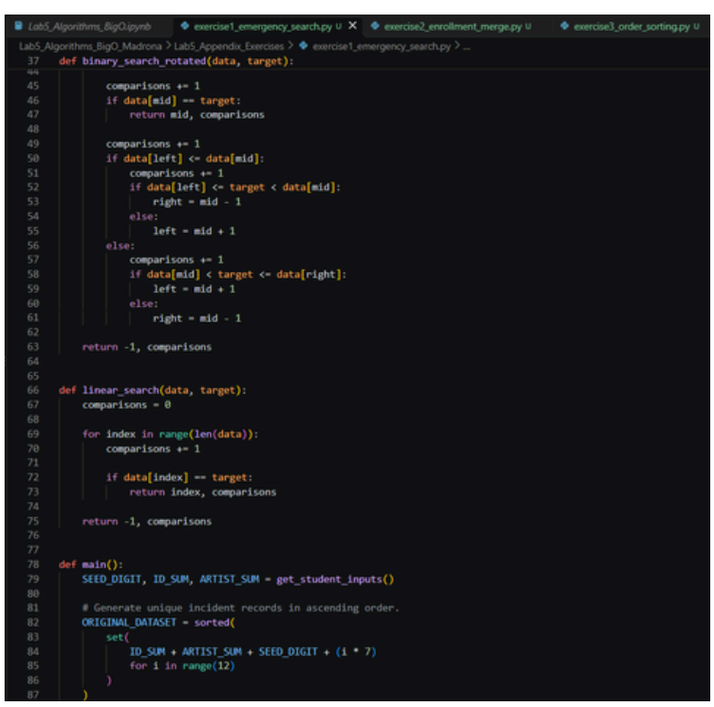
    print("\nOriginal Dataset:", ORIGINAL_DATASET)
    print("Rotation Point:", ROTATION_POINT)
    print("Rotated Dataset:", ROTATED_DATASET)
    print("Target Incident:", TARGET_INCIDENT)

    print("\n--- ROTATED BINARY SEARCH ---")

    if binary_index != -1:
        print("Target found at index:", binary_index)
    else:
        print("Target incident was not found.")

    print("Comparison Count:", binary_comparisons)

    print("\n--- LINEAR SEARCH ---")
```



```python
def binary_search_rotated(data, target):

        comparisons += 1
        if data[mid] == target:
            return mid, comparisons

        comparisons += 1
        if data[left] <= data[mid]:
            comparisons += 1
            if data[left] <= target < data[mid]:
                right = mid - 1
            else:
                left = mid + 1
        else:
            comparisons += 1
            if data[mid] < target <= data[right]:
                left = mid + 1
            else:
                right = mid - 1

    return -1, comparisons


def linear_search(data, target):
    comparisons = 0

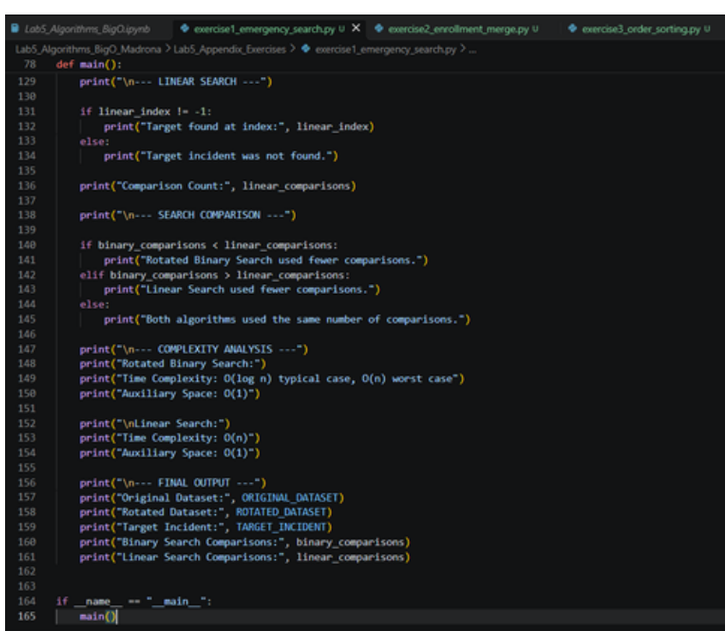
    for index in range(len(data)):
        comparisons += 1

        if data[index] == target:
            return index, comparisons

    return -1, comparisons


def main():
    SEED_DIGIT, ID_SUM, ARTIST_SUM = get_student_inputs()

    # Generate unique incident records in ascending order.
    ORIGINAL_DATASET = sorted(
        set(
            ID_SUM + ARTIST_SUM + SEED_DIGIT + (i * 7)
            for i in range(12)
        )
    )
```



```python
def main():
    print("\n--- LINEAR SEARCH ---")

    if linear_index != -1:
        print("Target found at index:", linear_index)
    else:
        print("Target incident was not found.")

    print("Comparison Count:", linear_comparisons)

    print("\n--- SEARCH COMPARISON ---")

    if binary_comparisons < linear_comparisons:
        print("Rotated Binary Search used fewer comparisons.")
    elif binary_comparisons > linear_comparisons:
        print("Linear Search used fewer comparisons.")
    else:
        print("Both algorithms used the same number of comparisons.")

    print("\n--- COMPLEXITY ANALYSIS ---")
    print("Rotated Binary Search:")
    print("Time Complexity: O(log n) typical case, O(n) worst case")
    print("Auxiliary Space: O(1)")

    print("\nLinear Search:")
    print("Time Complexity: O(n)")
    print("Auxiliary Space: O(1)")

    print("\n--- FINAL OUTPUT ---")
    print("Original Dataset:", ORIGINAL_DATASET)
    print("Rotated Dataset:", ROTATED_DATASET)
    print("Target Incident:", TARGET_INCIDENT)
    print("Binary Search Comparisons:", binary_comparisons)
    print("Linear Search Comparisons:", linear_comparisons)


if __name__ == "__main__":
    main()
```

```
Problems  Output  Debug Console  Terminal  Ports  Jupyter                    Python - Lab5_Algorithms_BigO_Madrona

(.venv) PS C:\Users\GLYDEL\OneDrive\MADRONA\Lab5_Algorithms_BigO_Madrona> & c:\Users\GLYDEL\OneDrive\MADRONA\Lab2_Fundamentals_Madrona\.venv\Scripts\p
ython.exe c:/Users/GLYDEL/OneDrive/MADRONA/Lab5_Algorithms_BigO_Madrona/Lab5_Appendix_Exercises/exercise1_emergency_search.py
Enter your last name: Madrona
Enter your student ID: TUPM-26-1133
Enter your favorite artist: The Ridleys

--- STUDENT-SPECIFIC INPUTS ---
LAST_NAME: Madrona
STUDENT_ID: TUPM-26-1133
FAVORITE_ARTIST: The Ridleys
SEED_DIGIT: 3
ID_SUM: 16
NAME_LENGTH: 7
ARTIST_LENGTH: 10
ARTIST_SUM: 125

==================================
EMERGENCY DISPATCH RECORD SEARCH
==================================

Original Dataset: [144, 151, 158, 165, 172, 179, 186, 193, 200, 207, 214, 221]
Rotation Point: 3
Rotated Dataset: [165, 172, 179, 186, 193, 200, 207, 214, 221, 144, 151, 158]
Target Incident: 207

--- ROTATED BINARY SEARCH ---
Target found at index: 6
Comparison Count: 7

--- LINEAR SEARCH ---
Target found at index: 6
Comparison Count: 7

--- SEARCH COMPARISON ---
Both algorithms used the same number of comparisons.

--- COMPLEXITY ANALYSIS ---
Rotated Binary Search:
Time Complexity: O(log n) typical case, O(n) worst case
```

```
Problems  Output  Debug Console  Terminal  Ports  Jupyter                    Python - Lab5_Algorithms_BigO_Madrona

(.venv) PS C:\Users\GLYDEL\OneDrive\MADRONA\Lab5_Algorithms_BigO_Madrona> & c:\Users\GLYDEL\OneDrive\MADRONA\Lab2_Fundamentals_Madrona\.venv\Scripts\p
ython.exe c:/Users/GLYDEL/OneDrive/MADRONA/Lab5_Algorithms_BigO_Madrona/Lab5_Appendix_Exercises/exercise1_emergency_search.py

==================================
EMERGENCY DISPATCH RECORD SEARCH
==================================

Original Dataset: [144, 151, 158, 165, 172, 179, 186, 193, 200, 207, 214, 221]
Rotation Point: 3
Rotated Dataset: [165, 172, 179, 186, 193, 200, 207, 214, 221, 144, 151, 158]
Target Incident: 207

--- ROTATED BINARY SEARCH ---
Target found at index: 6
Comparison Count: 7

--- LINEAR SEARCH ---
Target found at index: 6
Comparison Count: 7

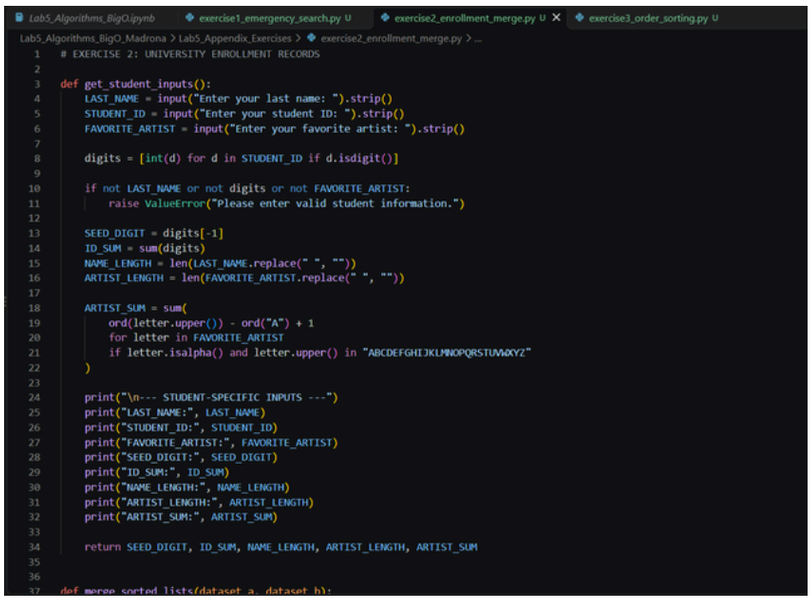
--- SEARCH COMPARISON ---
Both algorithms used the same number of comparisons.

--- COMPLEXITY ANALYSIS ---
Rotated Binary Search:
Time Complexity: O(log n) typical case, O(n) worst case
Auxiliary Space: O(1)

Linear Search:
Time Complexity: O(n)
Auxiliary Space: O(1)

--- FINAL OUTPUT ---
Original Dataset: [144, 151, 158, 165, 172, 179, 186, 193, 200, 207, 214, 221]
Rotated Dataset: [165, 172, 179, 186, 193, 200, 207, 214, 221, 144, 151, 158]
Target Incident: 207
Binary Search Comparisons: 7
Linear Search Comparisons: 7
(.venv) PS C:\Users\GLYDEL\OneDrive\MADRONA\Lab5_Algorithms_BigO_Madrona>
```

EXERCISE 2: UNIVERSITY ENROLLMENT RECORDS



```python
# EXERCISE 2: UNIVERSITY ENROLLMENT RECORDS

def get_student_inputs():
    LAST_NAME = input("Enter your last name: ").strip()
    STUDENT_ID = input("Enter your student ID: ").strip()
    FAVORITE_ARTIST = input("Enter your favorite artist: ").strip()

    digits = [int(d) for d in STUDENT_ID if d.isdigit()]

    if not LAST_NAME or not digits or not FAVORITE_ARTIST:
        raise ValueError("Please enter valid student information.")

    SEED_DIGIT = digits[-1]
    ID_SUM = sum(digits)
    NAME_LENGTH = len(LAST_NAME.replace(" ", ""))
    ARTIST_LENGTH = len(FAVORITE_ARTIST.replace(" ", ""))

    ARTIST_SUM = sum(
        ord(letter.upper()) - ord("A") + 1
        for letter in FAVORITE_ARTIST
        if letter.isalpha() and letter.upper() in "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    )

    print("\n--- STUDENT-SPECIFIC INPUTS ---")
    print("LAST_NAME:", LAST_NAME)
    print("STUDENT_ID:", STUDENT_ID)
    print("FAVORITE_ARTIST:", FAVORITE_ARTIST)
    print("SEED_DIGIT:", SEED_DIGIT)
    print("ID_SUM:", ID_SUM)
    print("NAME_LENGTH:", NAME_LENGTH)
    print("ARTIST_LENGTH:", ARTIST_LENGTH)
    print("ARTIST_SUM:", ARTIST_SUM)

    return SEED_DIGIT, ID_SUM, NAME_LENGTH, ARTIST_LENGTH, ARTIST_SUM


def merge_sorted_lists(dataset_a, dataset_b):
    i = 0
    j = 0
    merged = []
    comparisons = 0

    # Compare the current values from both datasets.
    while i < len(dataset_a) and j < len(dataset_b):
        comparisons += 1

        if dataset_a[i] <= dataset_b[j]:
            merged.append(dataset_a[i])
            i += 1
        else:
            merged.append(dataset_b[j])
            j += 1

    # Append the remaining records.
    merged.extend(dataset_a[i:])
    merged.extend(dataset_b[j:])

    return merged, comparisons


def main():
    SEED_DIGIT, ID_SUM, NAME_LENGTH, ARTIST_LENGTH, ARTIST_SUM = (
        get_student_inputs()
    )

    # Generate Department A records using SEED_DIGIT and ID_SUM.
    DATASET_A = sorted(
        set(SEED_DIGIT + ID_SUM + (i * 3) for i in range(8))
    )

    # Generate Department B records using ARTIST_LENGTH and NAME_LENGTH.
    DATASET_B = sorted(
        set(ARTIST_LENGTH + NAME_LENGTH + ARTIST_SUM + (i * 4)
            for i in range(7))
    )

    # Merge the two sorted datasets.
    CONSOLIDATED_DATASET, COMPARISON_COUNT = merge_sorted_lists(
        DATASET_A, DATASET_B
    )

    print("\n====================================")
    print("UNIVERSITY ENROLLMENT RECORDS")
    print("====================================")

    print("\nDepartment A Dataset:", DATASET_A)
    print("Department B Dataset:", DATASET_B)

    print("\n--- CONSOLIDATION RESULTS ---")
    print("Consolidated Dataset:", CONSOLIDATED_DATASET)
    print("Department A Record Count:", len(DATASET_A))
    print("Department B Record Count:", len(DATASET_B))

    print(
        "Records Before Consolidation:",
        len(DATASET_A) + len(DATASET_B)
    )
    print("Records After Consolidation:", len(CONSOLIDATED_DATASET))
    print("Comparison Count:", COMPARISON_COUNT)

    # Verify that the merged dataset is sorted.
    ORDERING_VERIFICATION = (
        CONSOLIDATED_DATASET == sorted(CONSOLIDATED_DATASET)
    )

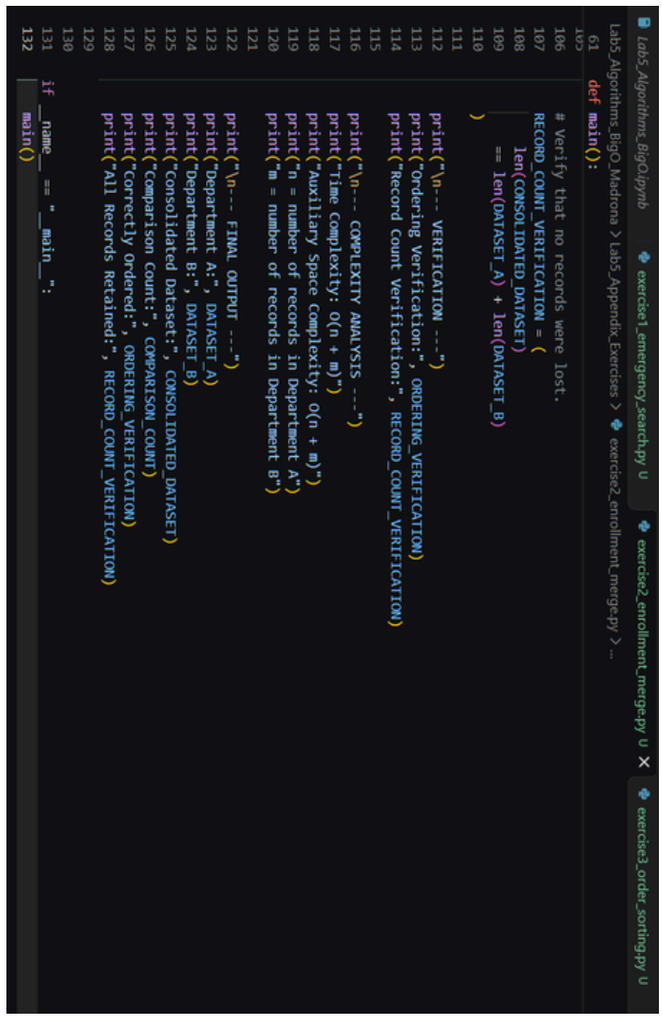
    # Verify that no records were lost.
    RECORD_COUNT_VERIFICATION = (
        len(CONSOLIDATED_DATASET)
        == len(DATASET_A) + len(DATASET_B)
    )

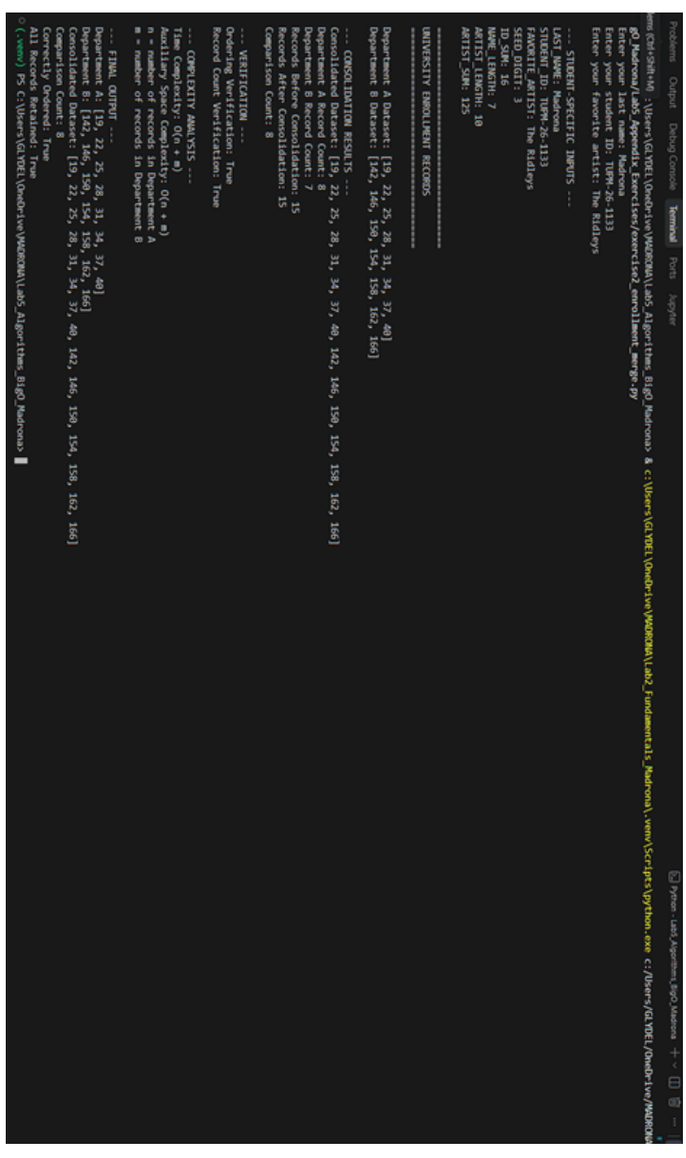
    print("\n--- VERIFICATION ---")
    print("Ordering Verification:", ORDERING_VERIFICATION)
    print("Record Count Verification:", RECORD_COUNT_VERIFICATION)

    print("\n--- COMPLEXITY ANALYSIS ---")
    print("Time Complexity: O(n + m)")
    print("Auxiliary Space Complexity: O(n + m)")
    print("n = number of records in Department A")
    print("m = number of records in Department B")

    print("\n--- FINAL OUTPUT ---")
    print("Department A:", DATASET_A)
    print("Department B:", DATASET_B)
    print("Consolidated Dataset:", CONSOLIDATED_DATASET)
    print("Comparison Count:", COMPARISON_COUNT)
    print("All Records Retained:", RECORD_COUNT_VERIFICATION)


if __name__ == "__main__":
    main()
```





```
(.venv) PS C:\Users\GLYDEL\OneDrive\MADRONA\Lab5_Algorithms_BigO_Madrona> & c:\Users\GLYDEL\OneDrive\MADRONA\Lab5_Fundamentals_Madrona\.venv\Scripts\python.exe c:/Users/GLYDEL/OneDrive/MADRONA/Lab5_Algorithms_BigO_Madrona/Lab5_Appendix_Exercises/exercise2_enrollment_merge.py
Enter your last name: Madrona
Enter your student ID: TUPM-26-1133
Enter your favorite artist: The Ridleys

--- STUDENT-SPECIFIC INPUTS ---
LAST_NAME: Madrona
STUDENT_ID: TUPM-26-1133
FAVORITE_ARTIST: The Ridleys
SEED_DIGIT: 3
ID_SUM: 16
NAME_LENGTH: 7
ARTIST_LENGTH: 10
ARTIST_SUM: 125

====================================
UNIVERSITY ENROLLMENT RECORDS
====================================

Department A Dataset: [19, 22, 25, 28, 31, 34, 37, 40]
Department B Dataset: [142, 146, 150, 154, 158, 162, 166]

--- CONSOLIDATION RESULTS ---
Consolidated Dataset: [19, 22, 25, 28, 31, 34, 37, 40, 142, 146, 150, 154, 158, 162, 166]
Department A Record Count: 8
Department B Record Count: 7
Records Before Consolidation: 15
Records After Consolidation: 15
Comparison Count: 8

--- VERIFICATION ---
Ordering Verification: True
Record Count Verification: True

--- COMPLEXITY ANALYSIS ---
Time Complexity: O(n + m)
Auxiliary Space Complexity: O(n + m)
n = number of records in Department A
m = number of records in Department B

--- FINAL OUTPUT ---
Department A: [19, 22, 25, 28, 31, 34, 37, 40]
Department B: [142, 146, 150, 154, 158, 162, 166]
Consolidated Dataset: [19, 22, 25, 28, 31, 34, 37, 40, 142, 146, 150, 154, 158, 162, 166]
Comparison Count: 8
All Records Retained: True
(.venv) PS C:\Users\GLYDEL\OneDrive\MADRONA\Lab5_Algorithms_BigO_Madrona>
```

EXERCISE 3: E-COMMERCE ORDER PROCESSING

```python
# EXERCISE 3: E-COMMERCE ORDER PROCESSING

import random
import time


def get_student_inputs():
    LAST_NAME = input("Enter your last name: ").strip()
    STUDENT_ID = input("Enter your student ID: ").strip()
    FAVORITE_ARTIST = input("Enter your favorite artist: ").strip()

    digits = [int(d) for d in STUDENT_ID if d.isdigit()]

    if not LAST_NAME or not digits or not FAVORITE_ARTIST:
        raise ValueError("Please enter valid student information.")

    SEED_DIGIT = digits[-1]
    ID_SUM = sum(digits)
    NAME_LENGTH = len(LAST_NAME.replace(" ", ""))
    ARTIST_LENGTH = len(FAVORITE_ARTIST.replace(" ", ""))

    ARTIST_SUM = sum(
        ord(letter.upper()) - ord("A") + 1
        for letter in FAVORITE_ARTIST
        if letter.isalpha() and letter.upper() in "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    )

    print("\n--- STUDENT-SPECIFIC INPUTS ---")
    print("LAST_NAME:", LAST_NAME)
    print("STUDENT_ID:", STUDENT_ID)
    print("FAVORITE_ARTIST:", FAVORITE_ARTIST)
    print("SEED_DIGIT:", SEED_DIGIT)
    print("ID_SUM:", ID_SUM)
    print("NAME_LENGTH:", NAME_LENGTH)
    print("ARTIST_LENGTH:", ARTIST_LENGTH)
    print("ARTIST_SUM:", ARTIST_SUM)

    return SEED_DIGIT, ID_SUM, NAME_LENGTH, ARTIST_LENGTH, ARTIST_SUM


# BUBBLE SORT
def bubble_sort(data):
    arr = data.copy()
    comparisons = 0
    movements = 0
```

```python
def main():
    # Generate the original order dataset using student inputs.
    base_orders = [
        ID_SUM + ARTIST_SUM + SEED_DIGIT + (i * 2)
        for i in range(12)
    ]

    # Use student-specific values to make the random dataset reproducible.
    random_seed = (
        ID_SUM + ARTIST_SUM + SEED_DIGIT
        + NAME_LENGTH + ARTIST_LENGTH
    )
    rng = random.Random(random_seed)

    # Create the three input conditions from the same orders.
    sorted_orders = sorted(base_orders)
    reverse_orders = sorted(base_orders, reverse=True)
    random_orders = rng.sample(base_orders, len(base_orders))

    datasets = {
        "SORTED INPUT": sorted_orders,
        "REVERSE-ORDERED INPUT": reverse_orders,
        "RANDOM INPUT": random_orders,
    }

    algorithms = {
        "Bubble Sort": bubble_sort,
        "Selection Sort": selection_sort,
        "Insertion Sort": insertion_sort,
    }

    print("\n" + "=" * 40)
    print("E-COMMERCE ORDER PROCESSING")
    print("=" * 40)
    print("Generated Order Dataset:", base_orders)

    results = []

    for condition, data in datasets.items():
        print("\n" + "-" * 40)
        print(condition)
        print("Input:", data)

        for algorithm_name, algorithm in algorithms.items():
```

```python
def bubble_sort(data):
    arr = data.copy()
    comparisons = 0
    movements = 0
    n = len(arr)

    for end in range(n - 1, 0, -1):
        swapped = False

        for i in range(end):
            comparisons += 1

            if arr[i] > arr[i + 1]:
                arr[i], arr[i + 1] = arr[i + 1], arr[i]
                movements += 1
                swapped = True

        if not swapped:
            break

    return arr, comparisons, movements


# SELECTION SORT
def selection_sort(data):
    arr = data.copy()
    comparisons = 0
    movements = 0
    n = len(arr)

    for i in range(n - 1):
        minimum = i

        for j in range(i + 1, n):
            comparisons += 1

            if arr[j] < arr[minimum]:
                minimum = j

        if minimum != i:
            arr[i], arr[minimum] = arr[minimum], arr[i]
            movements += 1

    return arr, comparisons, movements
```

```python
def main():
        for algorithm_name, algorithm in algorithms.items():
            sorted_data, comparisons, movements, elapsed = (
                measure_sorting(algorithm, data)
            )

            verified = sorted_data == sorted(data)

            print("\nAlgorithm:", algorithm_name)
            print("Sorted Output:", sorted_data)
            print("Comparisons:", comparisons)
            print("Movements:", movements)
            print(f"Execution Time: {elapsed:.8f} seconds")
            print("Sorting Verification:", verified)

            results.append(
                (
                    condition,
                    algorithm_name,
                    comparisons,
                    movements,
                    elapsed,
                    verified,
                )
            )

    print("\n" + "=" * 40)
    print("PERFORMANCE SUMMARY")
    print("=" * 40)
    print(
        f"{'Input Condition':<22}"
        f"{'Algorithm':<20}"
        f"{'Comparisons':<14}"
        f"{'Movements':<12}"
        f"{'Time (seconds)':<16}"
    )

    for condition, algorithm, comparisons, movements, elapsed, verified in results:
        print(
            f"{condition:<22}"
            f"{algorithm:<20}"
            f"{comparisons:<14}"
            f"{movements:<12}"
            f"{elapsed:<16.8f}"
```

```python
# INSERTION SORT
def insertion_sort(data):
    arr = data.copy()
    comparisons = 0
    movements = 0

    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0:
            comparisons += 1

            if arr[j] > key:
                arr[j + 1] = arr[j]
                movements += 1
                j -= 1
            else:
                break

        arr[j + 1] = key
        movements += 1

    return arr, comparisons, movements


# MEASURE EXECUTION TIME
def measure_sorting(algorithm, data):
    start_time = time.perf_counter()

    sorted_data, comparisons, movements = algorithm(data)

    end_time = time.perf_counter()
    execution_time = end_time - start_time

    return sorted_data, comparisons, movements, execution_time


def main():
    SEED_DIGIT, ID_SUM, NAME_LENGTH, ARTIST_LENGTH, ARTIST_SUM = (
        get_student_inputs()
    )
```

```python
def main():
        f"{'Time (seconds)':<16}"
    )

    for condition, algorithm, comparisons, movements, elapsed, verified in results:
        print(
            f"{condition:<22}"
            f"{algorithm:<20}"
            f"{comparisons:<14}"
            f"{movements:<12}"
            f"{elapsed:<16.8f}"
        )

    print("\n" + "=" * 40)
    print("BIG O ANALYSIS")
    print("=" * 40)
    print("Bubble Sort:")
    print("Best Case: O(n)")
    print("Average Case: O(n^2)")
    print("Worst Case: O(n^2)")

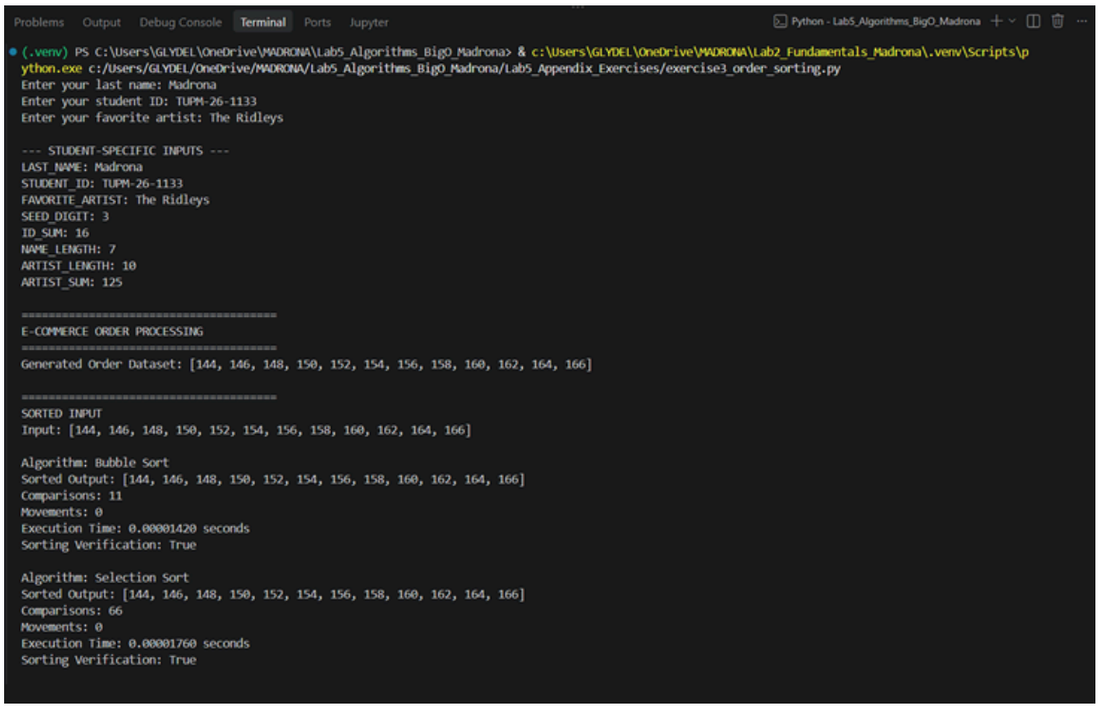
    print("\nSelection Sort:")
    print("Best Case: O(n^2)")
    print("Average Case: O(n^2)")
    print("Worst Case: O(n^2)")

    print("\nInsertion Sort:")
    print("Best Case: O(n)")
    print("Average Case: O(n^2)")
    print("Worst Case: O(n^2)")

    print("\nAuxiliary Space:")
    print("Each algorithm uses O(n) space here because it sorts a copy.")
    print("The sorting operations themselves use O(1) extra space.")

    print("\nAll sorting results verified:",
          all(result[5] for result in results))


if __name__ == "__main__":
    main()
```



```
Problems  Output  Debug Console  Terminal  Ports  Jupyter                    Python - Lab5_Algorithms_BigO_Madrona

(.venv) PS C:\Users\GLYDEL\OneDrive\MADRONA\Lab5_Algorithms_BigO_Madrona> & c:\Users\GLYDEL\OneDrive\MADRONA\Lab2_Fundamentals_Madrona\.venv\Scripts\p
ython.exe c:/Users/GLYDEL/OneDrive/MADRONA/Lab5_Algorithms_BigO_Madrona/Lab5_Appendix_Exercises/exercise3_order_sorting.py
Enter your last name: Madrona
Enter your student ID: TUPM-26-1133
Enter your favorite artist: The Ridleys

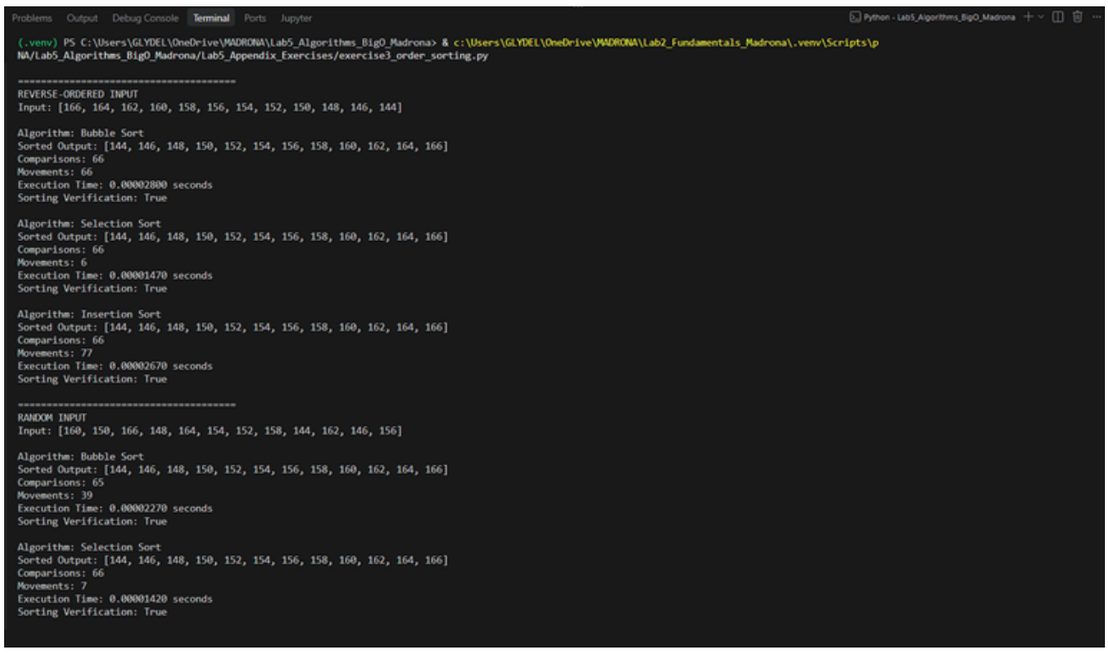
--- STUDENT-SPECIFIC INPUTS ---
LAST_NAME: Madrona
STUDENT_ID: TUPM-26-1133
FAVORITE_ARTIST: The Ridleys
SEED_DIGIT: 3
ID_SUM: 16
NAME_LENGTH: 7
ARTIST_LENGTH: 10
ARTIST_SUM: 125

==================================
E-COMMERCE ORDER PROCESSING
==================================
Generated Order Dataset: [144, 146, 148, 150, 152, 154, 156, 158, 160, 162, 164, 166]

==================================
SORTED INPUT
Input: [144, 146, 148, 150, 152, 154, 156, 158, 160, 162, 164, 166]

Algorithm: Bubble Sort
Sorted Output: [144, 146, 148, 150, 152, 154, 156, 158, 160, 162, 164, 166]
Comparisons: 11
Movements: 0
Execution Time: 0.00001420 seconds
Sorting Verification: True

Algorithm: Selection Sort
Sorted Output: [144, 146, 148, 150, 152, 154, 156, 158, 160, 162, 164, 166]
Comparisons: 66
Movements: 0
Execution Time: 0.00001760 seconds
Sorting Verification: True
```



```
Problems  Output  Debug Console  Terminal  Ports  Jupyter                    Python - Lab5_Algorithms_BigO_Madrona

(.venv) PS C:\Users\GLYDEL\OneDrive\MADRONA\Lab5_Algorithms_BigO_Madrona> & c:\Users\GLYDEL\OneDrive\MADRONA\Lab2_Fundamentals_Madrona\.venv\Scripts\p
NA/Lab5_Algorithms_BigO_Madrona/Lab5_Appendix_Exercises/exercise3_order_sorting.py

==================================
REVERSE-ORDERED INPUT
Input: [166, 164, 162, 160, 158, 156, 154, 152, 150, 148, 146, 144]

Algorithm: Bubble Sort
Sorted Output: [144, 146, 148, 150, 152, 154, 156, 158, 160, 162, 164, 166]
Comparisons: 66
Movements: 66
Execution Time: 0.00002800 seconds
Sorting Verification: True

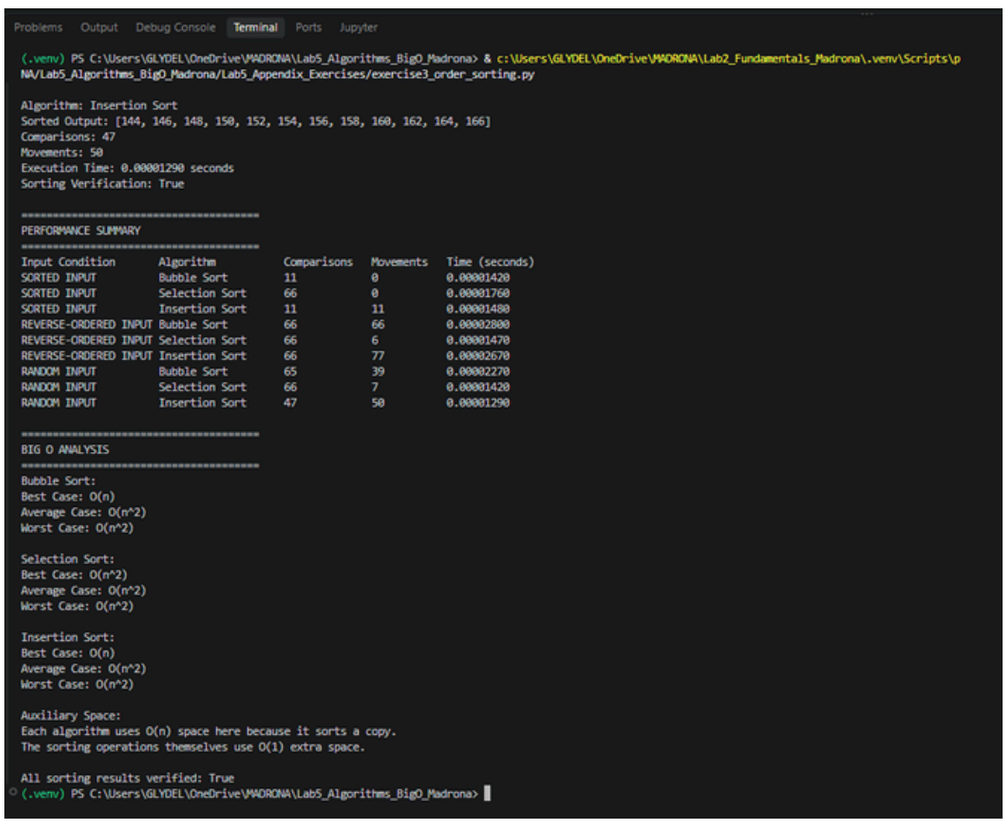
Algorithm: Selection Sort
Sorted Output: [144, 146, 148, 150, 152, 154, 156, 158, 160, 162, 164, 166]
Comparisons: 66
Movements: 6
Execution Time: 0.00001470 seconds
Sorting Verification: True

Algorithm: Insertion Sort
Sorted Output: [144, 146, 148, 150, 152, 154, 156, 158, 160, 162, 164, 166]
Comparisons: 66
Movements: 77
Execution Time: 0.00002670 seconds
Sorting Verification: True

==================================
RANDOM INPUT
Input: [160, 150, 166, 148, 164, 154, 152, 158, 144, 162, 146, 156]

Algorithm: Bubble Sort
Sorted Output: [144, 146, 148, 150, 152, 154, 156, 158, 160, 162, 164, 166]
Comparisons: 65
Movements: 39
Execution Time: 0.00002270 seconds
Sorting Verification: True

Algorithm: Selection Sort
Sorted Output: [144, 146, 148, 150, 152, 154, 156, 158, 160, 162, 164, 166]
Comparisons: 66
Movements: 7
Execution Time: 0.00001420 seconds
Sorting Verification: True
```



```
Problems  Output  Debug Console  Terminal  Ports  Jupyter

(.venv) PS C:\Users\GLYDEL\OneDrive\MADRONA\Lab5_Algorithms_BigO_Madrona> & c:\Users\GLYDEL\OneDrive\MADRONA\Lab2_Fundamentals_Madrona\.venv\Scripts\p
NA/Lab5_Algorithms_BigO_Madrona/Lab5_Appendix_Exercises/exercise3_order_sorting.py

Algorithm: Insertion Sort
Sorted Output: [144, 146, 148, 150, 152, 154, 156, 158, 160, 162, 164, 166]
Comparisons: 47
Movements: 50
Execution Time: 0.00001290 seconds
Sorting Verification: True

==================================
PERFORMANCE SUMMARY
==================================
Input Condition       Algorithm          Comparisons   Movements   Time (seconds)
SORTED INPUT          Bubble Sort        11            0           0.00001420
SORTED INPUT          Selection Sort     66            0           0.00001760
SORTED INPUT          Insertion Sort     11            11          0.00001480
REVERSE-ORDERED INPUT Bubble Sort        66            66          0.00002800
REVERSE-ORDERED INPUT Selection Sort     66            6           0.00001470
REVERSE-ORDERED INPUT Insertion Sort     66            77          0.00002670
RANDOM INPUT          Bubble Sort        65            39          0.00002270
RANDOM INPUT          Selection Sort     66            7           0.00001420
RANDOM INPUT          Insertion Sort     47            50          0.00001290

==================================
BIG O ANALYSIS
==================================
Bubble Sort:
Best Case: O(n)
Average Case: O(n^2)
Worst Case: O(n^2)

Selection Sort:
Best Case: O(n^2)
Average Case: O(n^2)
Worst Case: O(n^2)

Insertion Sort:
Best Case: O(n)
Average Case: O(n^2)
Worst Case: O(n^2)

Auxiliary Space:
Each algorithm uses O(n) space here because it sorts a copy.
The sorting operations themselves use O(1) extra space.

All sorting results verified: True
(.venv) PS C:\Users\GLYDEL\OneDrive\MADRONA\Lab5_Algorithms_BigO_Madrona>
```